



BÀI THỰC HÀNH SỐ 1
Môn: MẬT MÃ & AN NINH MẠNG

-o0o-

Họ tên: Nguyễn Tấn Tài

MSSV: 2212990

Nhóm: L01

Phần 1. Các hệ mã đối xứng truyền thống

Câu 1: Cho một bản mã (ciphertext) được tạo ra bằng cách dùng hệ mã Caesar để mã hoá một văn bản viết bằng tiếng Anh, hãy giải mã ciphertext trên mà không cần biết thông tin khoá và giải thích cách làm, từ đó cho biết điểm yếu của giải thuật Caesar là gì?

Ciphertext:

KNXNMNSLKWJXMBFYJWGSIXFIRNYXBTWIKNXMWFSITAJWMJQRNSLFSIDFD

Trả lời:

Bước 1: Phân tích tần suất

- Trước tiên, đếm tần suất xuất hiện của từng chữ cái trong bản mã, so sánh tần suất này với bảng tần suất chữ cái tiếng Anh.
- Trong Tiếng anh, chữ cái "E" có tần suất cao nhất, tiếp theo là các chữ "T", "A", "O", "I", và "N". Ta có thể giả định rằng ký tự xuất hiện nhiều nhất trong bản mã tương ứng với chữ "E".

Bước 2: Giả định giá trị của khóa

- Giả định rằng ký tự có tần suất xuất hiện cao nhất trong bản mã là do chữ "E" trong văn bản gốc dịch chuyển thành.
- Từ đó, tính giá trị dịch chuyển (khóa) bằng cách xác định khoảng cách giữa ký tự trong bản mã và chữ "E".
- Áp dụng dịch chuyển này cho tất cả các ký tự trong bản mã để kiểm tra xem có nhận được văn bản có ý nghĩa không.

Bước 3: Thử các dịch chuyển khác

- Nếu giả định đầu tiên không tạo ra kết quả có ý nghĩa, ta tiếp tục thử dịch chuyển ký tự có tần suất cao nhất trong bản mã sang các ký tự khác trong bảng tần suất.

Kết quả

```
PS D:\HCMUT\HK241\C03069_Cryptography-and-Network-Security\Lab_1\src\Excercise_01> go run .\main.go
Top 5 characters with highest frequency:
Character: 'J', Frequency: 5
Character: 'X', Frequency: 5
Character: 'N', Frequency: 5
Character: 'F', Frequency: 5
Character: 'W', Frequency: 5
Top 5 characters with highest frequency and assumed decryption:
Assuming 'J' is 'E' (shift 5):
Decrypted text: FISHINGFRESHWATERBENDSADMITSWORDFISHRANDOMOVERHELMINGANYDAY

Assuming 'X' is 'E' (shift 19):
Decrypted text: RUETUZRSDQETIMFQDNQZPEMPYUFEIADPRUETDMZPAHQDTQXYUZSMZKPMK

Assuming 'N' is 'E' (shift 9):
Decrypted text: BEODEJCBNAODSWPANXAJZOWZIEPOSKNZBEODNWJZKRANDAHIEJCHJUZWU

Assuming 'F' is 'E' (shift 1):
Decrypted text: JMWLMRKJVIWLAEXIVFIRHWEHQMXWASVHJMWLVERHSZIVLIPQMRKERCHEC

Assuming 'W' is 'E' (shift 18):
Decrypted text: SVFUVATSERFUJNGREORAQFNQZVGFJBEQSVFUENABIREURYZVATNALQNL
```

Ta có thể thấy rằng ngay ở ngay chuỗi đầu tiên, giả sử ký tự J là ký tự E được mã hóa và chuyển dịch lại đoạn ciphertext, ta được một chuỗi kết hợp lại một câu có ý nghĩa:

Plaintext

FISHING FRESH WATER BENDS ADMITS WORD FISH RANDOM
OVERWHELMING ANY DAY

Điểm yếu của giải thuật Ceaser

Caesar Cipher chỉ có 25 khóa khả thi (dịch chuyển từ 1 đến 25). Điều này có nghĩa là kẻ tấn công có thể thử tất cả các dịch chuyển một cách dễ dàng (brute-force) để tìm ra văn bản gốc mà không cần nhiều công sức hay thời gian.

Hệ mã Caesar không làm thay đổi tần suất xuất hiện của các ký tự. Trong tiếng Anh, các chữ cái như "E", "T", "A" xuất hiện nhiều nhất. Kẻ tấn công có thể phân tích tần suất ký tự trong bản mã và so sánh với tần suất của ngôn ngữ để tìm ra khóa.

Khi mã hóa các văn bản dài hoặc có cấu trúc lặp lại, Caesar Cipher không thay đổi sự lặp lại này trong bản mã. Điều này giúp kẻ tấn công dễ dàng nhận diện các mẫu lặp và từ đó tìm ra khóa.

Caesar Cipher đặc biệt yếu khi sử dụng cho các ngôn ngữ tự nhiên (như tiếng Anh), vì các ngôn ngữ này có mẫu tần suất ký tự đặc trưng. Một hệ mã an toàn cần phá vỡ các mẫu này để tránh bị phân tích tần suất, nhưng Caesar Cipher lại không làm được điều đó.

Câu 2: Cho ciphertext: asvphgyt đã được mã hoá bằng hệ mã thay thế theo công thức:

$$C = (M + K) \bmod 26$$

Trong đó C là bản mã (ciphertext), M là thông điệp (plaintext), K là khóa. Hãy tìm lại khóa K và giải mã thông điệp?

Trả lời:

Vì không biết thông điệp gốc (plaintext) và cũng không có thêm thông tin gì về khóa, ta sẽ thử giải bài toán này bằng cách brute-force – thử các giá trị khác nhau cho K từ 0 đến 25 và kiểm tra xem kết quả có tạo thành một văn bản có ý nghĩa hay không.

Bước 1: Chuyển đổi ký tự sang số. Đầu tiên, ta sẽ chuyển đổi các ký tự trong bản mã thành các giá trị số (0 cho "a", 1 cho "b", ..., 25 cho "z").

Bước 2: Thử tất cả các giá trị của K (từ 0 đến 25)

Dùng công thức giải mã để tìm M từ C: $M = (C - K) \bmod 26$. Ta sẽ thử từng giá trị K từ 0 đến 25 và chuyển đổi các giá trị thu được từ M trở lại ký tự để xem văn bản có ý nghĩa không.

```
func main() {
    ciphertext := "asvphgyt"

    numericSlice := toNumeric(ciphertext)
    fmt.Println(numericSlice)

    fmt.Println("Testing all possible keys:")
    for key := 0; key < 26; key++ {
        plaintext := decrypt(numericSlice, key)
        fmt.Printf("Key %d: %s\n", key, plaintext)
    }
}
```

```
Key 22: ewztlkcx
Key 23: dvyskjbw
Key 24: cuxrjiav
Key 25: btwqihzu
```

```
Testing all possible keys:
Key 0: asvphgyt
Key 0: asvphgyt
Key 1: zruogfxs
Key 1: zruogfxs
Key 2: yqtnfewr
Key 3: xpsmedvq
Key 2: yqtnfewr
Key 3: xpsmedvq
Key 4: worldcup
Key 3: xpsmedvq
Key 4: worldcup
Key 5: vnqkcbto
Key 4: worldcup
Key 5: vnqkcbto
Key 6: umpjbasn
Key 6: umpjbasn
Key 7: tloiazrm
```

```
Key 8: sknhzyql
Key 8: sknhzyql
Key 9: rjmgypxpk
Key 9: rjmgypxpk
Key 10: qilfxwoj
Key 11: phkewvni
Key 12: ogjdvumh
Key 13: nficutlg
Key 14: mehbtskf
Key 15: ldgasrje
Key 16: kcfzrqid
Key 17: jbeyqphc
Key 18: iadxpogb
Key 19: hzcwonfa
Key 20: gybvrmz
Key 21: fxaumldy
```

Kết quả: Như ta đã thấy kết quả ở trên, thông qua brute-force, ta nhận thấy ở key 4, tức là chuyển định bản mã theo 4 đơn vị, ta được kết quả là một thông điệp gốc có nghĩa

Plaintext: WorldCup

Câu 3: Cho một bản mã (ciphertext) được tạo ra bởi mật mã Affine với công thức như sau (bản rõ là một văn bản tiếng Anh):

$$C = E([a, b], p) = (ap+b) \bmod 26$$

Người ta nhận thấy rằng trong bản mã này, ký tự B xuất hiện nhiều nhất, ký tự U xuất hiện nhiều thứ hai, hãy tìm ra công thức đúng của hệ mã Affine nói trên (tìm giá trị a, b)?

Trả lời:

Dựa vào bảng tần suất xuất hiện của các ký tự trong tiếng anh:

- Ký tự 'B' xuất hiện nhiều nhất: Điều này cho thấy rằng 'B' có khả năng được mã hóa từ chữ cái phổ biến nhất trong tiếng Anh, đó là 'E'.
- Ký tự 'U' xuất hiện nhiều thứ hai: Điều này cho thấy 'U' có khả năng được mã hóa từ chữ cái phổ biến thứ hai trong tiếng Anh, đó là 'T'.

Công thức mã hóa Affine: $C = E([a, b], p) = (ap+b) \bmod 26$

Trong đó:

- C là ký tự trong bản mã
- p là ký tự trong văn bản gốc
- a và b là các tham số của hệ mã Affine cần tìm,
- a và 26 phải nguyên tố cùng nhau ($\gcd(a, 26) = 1$), vì a phải có nghịch đảo modulo 26.

Các bước giải bài toán

Bước 1: Chuyển đổi ký tự sang giá trị số

- Ký tự 'B' trong bản mã là C1=1(vì 'B' là ký tự thứ hai trong bảng chữ cái, bắt đầu từ 0).
- Ký tự 'U' trong bản mã là C2=20.
- Ký tự 'E' trong văn bản gốc là P1=4(vì 'E' là ký tự thứ năm).
- Ký tự 'T' trong văn bản gốc là P2=19.

Bước 2: Thiết lập hai phương trình

Từ công thức mã hóa, ta có hai phương trình:

$$(axP_1 + b) \bmod 26 = C_1$$

$$(axP_2 + b) \bmod 26 = C_2$$

Thay giá trị vào:

$$(4a + b) \bmod 26 = 1$$

$$(19a + b) \bmod 26 = 20$$

Bước 3: Giải hệ phương trình để tìm a và b

Trừ hai phương trình:

$$(19a + b) - (4a + b) \equiv 20 - 1 \pmod{26}$$

$$15a \equiv 19 \pmod{26}$$

Để tìm a, ta cần tìm nghịch đảo modulo 26 của 15, tức là tìm số x sao cho:

$$15a \equiv 1 \pmod{26}$$

Tìm nghịch đảo của 15 modulo 26: sử dụng thuật toán Euclidean mở rộng, ta tìm được nghịch đảo của 15 modulo 26 là $x=7$. Nhân cả hai vế của phương trình $15a \equiv 19 \pmod{26}$ với 7

$$a \equiv (19 \times 7) \pmod{26}$$

$$a \equiv 3 \pmod{26}$$

Vậy, $a = 3$.

Thay $a = 3$ vào phương trình đầu tiên

$$12 + b \equiv 1 \pmod{26}$$

$$b \equiv 15 \pmod{26}$$

Vậy, $b=15$.

Công thức mã hóa Affine cho bản mã này là:

$$C = (3p+15) \bmod 26$$

Câu 4: Hãy nêu ra hai vấn đề đối với Mật mã One-time Pad?

1. Quản lý và phân phối khóa

OTP yêu cầu khóa phải có độ dài bằng với thông điệp: Mỗi ký tự trong thông điệp cần một ký tự trong khóa tương ứng để mã hóa, vì vậy độ dài khóa phải tương đương với độ dài của dữ liệu. Đối với các thông điệp lớn, việc tạo ra và lưu trữ khóa lớn tương ứng là một thách thức.

Khóa phải được phân phối an toàn: Để đảm bảo tính bảo mật, khóa cần được gửi tới người nhận một cách bảo mật, nhưng quá trình này khó thực hiện nếu người gửi và người nhận không gặp trực tiếp. Nếu khóa bị đánh cắp trong quá trình phân phối, tính bảo mật của OTP bị phá vỡ hoàn toàn.

2. Không thể tái sử dụng khóa

Khóa chỉ có thể sử dụng một lần duy nhất: Nếu khóa được tái sử dụng cho nhiều thông điệp, hệ thống mã hóa sẽ trở nên dễ bị tấn công bởi các phương pháp phân tích tần suất và phân tích chồng lấp (XOR hai bản mã sẽ tạo ra các mẫu có thể tiết lộ thông tin về văn bản gốc). Điều này có nghĩa là cần tạo ra một khóa mới, ngẫu nhiên và duy nhất cho mỗi lần truyền dữ liệu.

Tốn kém tài nguyên và phức tạp: Việc tạo ra một khóa ngẫu nhiên cho mỗi thông điệp và đảm bảo nó không bị tái sử dụng đòi hỏi tài nguyên và hệ thống quản lý khóa phức tạp. Điều này đặc biệt khó khăn trong các hệ thống lớn hoặc khi cần trao đổi thông tin thường xuyên.

Câu 5: Hãy sử dụng mật mã Playfair để mã hoá thông điệp bên dưới:

Must see you over Cadogan West. Coming at once.

M	F	H	I/J	K
U	N	O	P	Q
Z	V	W	X	Y
E	L	A	R	G
D	S	T	B	C

Bước 1: Chuẩn bị thông điệp

- Loại bỏ dấu câu và chuyển thành chữ thường: "must see you over cadogan west coming at once"

- Chia thành từng cặp chữ cái: Nếu có hai chữ cái giống nhau trong một cặp, chèn chữ "X" giữa chúng, nếu còn một chữ cái lẻ cuối cùng, thêm "X" vào cuối để đảm bảo mỗi cặp đều có hai ký tự.

MU ST SE EY OU OV ER CA DO GA NW ES TC OM IN GA TO NC EX

Bước 2: Sử dụng ma trận khóa để mã hóa từng cặp

Ma trận khóa Playfair đã cho, quy tắc mã hóa Playfair

- Nếu hai chữ cái nằm cùng hàng: Thay mỗi chữ cái bằng chữ cái nằm bên phải nó (quay về đầu hàng nếu là ký tự cuối).
- Nếu hai chữ cái nằm cùng cột: Thay mỗi chữ cái bằng chữ cái nằm bên dưới nó (quay lên đầu cột nếu là ký tự cuối).
- Nếu hai chữ cái nằm khác hàng và cột: Thay mỗi chữ cái bằng chữ cái ở góc đối diện trong hình chữ nhật tạo bởi hai chữ cái đó.

UN TB TL ZG PN NW LG TG UM ER OV LD BD NU FB ER DP NC EX

Câu 6: Sử dụng mật mã Double Transposition để mã hoá thông điệp bên dưới, mô tả từng bước thực hiện? Plaintext: "spyarrivesonthursday". Key1 = TAN, Key2 = TAI.

Bước 1: Chuẩn bị kích thước lưới và điền văn bản vào lưới

- Đếm số ký tự trong văn bản: Văn bản "spyarrivesonthursday" có 19 ký tự.
- Xác định kích thước lưới: Kích thước lưới phù hợp sẽ là 5x4, vì 5 hàng x 4 cột = 20 ô. Lưới này có 1 ô thừa (20 - 19 = 1), nên chúng ta cần thêm một ký tự giả vào cuối để làm đầy lưới. Giả sử chúng ta thêm ký tự "X" vào cuối văn bản. Vậy, văn bản mới để mã hóa sẽ là "spyarrivesonthursdayx".
- Điền các ký tự vào lưới 5x4 theo thứ tự từ trái qua phải và từ trên xuống dưới như sau:

s	p	y	a
r	r	i	v
e	s	o	n
t	h	u	r
s	d	a	y
x			

Bước 2: Thực hiện hoán vị hàng với khóa key1 = "TAN"

- Chuyển đổi khóa key1 thành thứ tự sắp xếp hàng: Với key1 = "TAN", ta sắp xếp các chữ cái theo thứ tự bảng chữ cái: "A" đứng thứ 1, "N" đứng thứ 2, và "T" đứng thứ 3; Vì vậy, thứ tự sắp xếp của các hàng là [2, 3, 1, 4, 5].
- Sắp xếp lại hàng: Sắp xếp các hàng trong lưới theo thứ tự [2, 3, 1, 4, 5] như sau:

r r i v (Hàng 2)
e s o n (Hàng 3)
s p y a (Hàng 1)
t h u r (Hàng 4)
s d a y (Hàng 5)

Bước 3: Thực hiện hoán vị cột với khóa key2 = "TAI"

- Chuyển đổi khóa key2 thành thứ tự sắp xếp cột: Với key2 = "TAI", ta sắp xếp các chữ cái theo thứ tự bảng chữ cái: "A" đứng thứ 1, "I" đứng thứ 2, và "T" đứng thứ 3; Thứ tự cột sau khi sắp xếp là [3, 1, 2, 4].
- Sắp xếp lại các cột: Thay đổi thứ tự cột trong lưới theo thứ tự [3, 1, 2, 4]

i r r v
o e s n
y s p a
u t h r
a s d y

Bước 4: Đọc bản mã

Sau khi thực hiện hoán vị hàng và cột, đọc các ký tự theo thứ tự hàng từ trên xuống dưới và từ trái qua phải để tạo thành bản mã cuối cùng.

Plaintext: **irrv oesn yspa uthr asdy**

Phần 2. Chuẩn mã hoá dữ liệu DES

Câu 1:

1. Mã hóa khối (Block Cipher)

Mã hóa khối xử lý dữ liệu theo từng khối cố định có kích thước nhất định (thường là 64 hoặc 128 bit). Mỗi khối dữ liệu được mã hóa thành một khối mã hóa có cùng kích thước. Mỗi khối dữ liệu được xử lý độc lập nhằm đạt được độ bảo mật cao nhất.

Cách thức hoạt động

Dữ liệu được chia thành nhiều khối, và mỗi khối sẽ được mã hóa riêng biệt theo một thuật toán mã hóa nhất định, như DES hoặc AES. Nếu khối cuối cùng không đủ độ dài yêu cầu, nó có thể cần được thêm vào (padding).

Mã hóa khối có nhiều chế độ hoạt động khác nhau như ECB (Electronic Code Book), CBC (Cipher Block Chaining), CFB (Cipher Feedback), v.v. Mỗi chế độ có cách xử lý khối khác nhau để tăng tính bảo mật.

Nhận xét

Nhược điểm: Chậm hơn mã hóa dòng trong một số trường hợp, vì phải xử lý từng khối một. Trong một vài trường hợp, có thể yêu cầu thêm dữ liệu (padding) nếu khối cuối không đủ kích thước.

Ưu điểm: Bảo mật tốt khi mã hóa các khối dữ liệu lớn. Các khối được mã hóa độc lập với nhau (tùy vào chế độ hoạt động), dễ kiểm soát và dễ quản lý.

2. Mã hóa dòng (Stream Cipher)

Mã hóa dòng xử lý dữ liệu theo từng bit hoặc từng byte của dòng dữ liệu liên tục, thay vì theo khối. Mã hóa dòng sử dụng một khóa và một bộ tạo keystream (dòng khóa) để mã hóa từng phần dữ liệu một cách liên tục.

Cách thức hoạt động

Tạo Keystream - một chuỗi bit được tạo từ khóa ban đầu và dùng để XOR với từng bit hoặc từng byte của dữ liệu gốc để tạo ra bản mã. Dữ liệu được mã hóa trực tiếp mà không cần chia thành các khối cố định hoặc thêm padding.

Nhận xét

Ưu điểm: Nhanh hơn mã hóa khối vì không cần chia dữ liệu thành từng khối. Hiệu quả hơn khi mã hóa loại dữ liệu liên tục hoặc kích thước nhỏ.

Nhược điểm: Khó kiểm soát và quản lý keystream, dễ gặp phải các vấn đề bảo mật nếu keystream bị lặp lại. Việc lưu trữ và bảo mật keystream phá sinh ra rất nhiều bài toán nan giải cần phải xử lý song song với việc quản lý. Nhạy cảm với sự đồng bộ giữa bên gửi và bên nhận, vì một sự thay đổi nhỏ có thể ảnh hưởng đến toàn bộ bản mã.

	Block Cipher	Stream Cipher
Phương pháp	Mã hóa theo khối cố định	Mã hóa từng bit hoặc từng byte
Chia thành blocks	Có	Không
Tốc độ	Chậm hơn	Nhanh hơn
Padding data	Có	Không
Thuật toán	DES, AES	RC4, A5/1
Phù hợp	Dữ liệu có kích thước lớn	Dữ liệu liên tục, thời gian thực

Câu 2:

Plaintext: 0 1 2 3 4 5 6 7 8 9 A B C D E F

Key: 0 1 2 3 4 5 6 7 8 9 A B 2 9 9 0

a. Tính khoá con K1 được sử dụng cho vòng mã hoá đầu tiên

Bước 1: Chuyển khóa từ dạng hex sang nhị phân

0 --> 0000	8 --> 1000
1 --> 0001	9 --> 1001
2 --> 0010	A --> 1010
3 --> 0011	B --> 1011
4 --> 0100	2 --> 0010
5 --> 0101	9 --> 1001
6 --> 0110	9 --> 1001
7 --> 0111	0 --> 0000

Vậy, khóa ở dạng nhị phân sẽ là:

K = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 0010 1001 1001 0000

Bước 2: Loại bỏ các bit chẵn lẻ

Trong DES, khóa chính 64-bit được chuyển thành 56-bit bằng cách loại bỏ các bit kiểm tra chẵn lẻ (các bit ở vị trí 8, 16, 24, 32, 40, 48, 56, và 64). Sau khi loại bỏ các bit chẵn lẻ, ta còn lại khóa 56-bit như sau:

K₊ = 000000 010010 001101 000101 010110 011110 001010 101100 101001 1001 100000

Bước 3: Chia khóa thành hai nửa **C₀** và **D₀**

Khóa 56-bit sẽ được chia thành hai phần, mỗi phần 28-bit:

- C₀: 28bit đầu tiên. **C₀ = 000000 010010 001101 000101 010110 01**
- D₀: 28bit cuối cùng. **D₀ = 011110 001010 101100 101001 100110**

Bước 4: Dịch vòng trái để tạo khóa **C₁** và **D₁**

Theo dịch vòng của DES, vòng đầu tiên yêu cầu dịch trái 1bit cho cả **C₀** và **D₀**.

C₁ = 000000 100100 011010 001010 101100 10

D₁ = 111100 010101 011001 010011 001101

Bước 5: Áp dụng hoán vị PC-2 để tạo khóa con **K₁**

Sau khi có **C₁** và **D₁**, chúng ta ghép chúng lại thành một khóa 56-bit và áp dụng bảng PC-2 để hoán vị và tạo khóa con **K₁**.

Bảng PC-2 – được sử dụng cố định trong thuật toán mã hóa DES

14 17 11 24 1 5

3 28 15 6 21 10

23 19 12 4 26 8

16 7 27 20 13 2

41 52 31 37 47 55

30 40 51 45 33 48

44 49 39 56 34 53

46 42 50 36 29 32

Áp dụng PC-2 lên **C₁** và **D₁**, ta nhận được khóa con **K₁** (48 bit):

K₁ = 000110 110000 001011 101111 111111 000111 000001 110010

b. Tính L0, R0

Bước 1: Chuyển đổi M từ hex sang nhị phân

0 --> 0000	8 --> 1000
1 --> 0001	9 --> 1001
2 --> 0010	A --> 1010
3 --> 0011	B --> 1011
4 --> 0100	C --> 1100
5 --> 0101	D --> 1101
6 --> 0110	E --> 1110
7 --> 0111	F --> 1111

Vậy, M ở dạng nhị phân sẽ là:

M = 00000001 00100011 01000101 01100111 10001001 10101011 11001101 11101111

Bước 2: Áp dụng Hoán vị Khởi đầu (Initial Permutation - IP)

Bảng IP trong DES được cố định và có tác dụng sắp xếp lại các bit của M để tạo ra một chuỗi bit mới.

Bảng IP:

```

58 50 42 34 26 18 10 2
60 52 44 36 28 20 12 4
62 54 46 38 30 22 14 6
64 56 48 40 32 24 16 8
57 49 41 33 25 17 9 1
59 51 43 35 27 19 11 3
61 53 45 37 29 21 13 5
63 55 47 39 31 23 15 7

```

Mỗi số trong bảng IP chỉ định vị trí của bit trong M để tạo ra khối IP mới. Ví dụ, bit thứ 58 của M sẽ là bit đầu tiên trong kết quả IP, bit thứ 50 của M sẽ là bit thứ hai, v.v.

IP(M) = 11001100 00000000 11001100 11111111 11110000 10101010 11110000 10101010

Bước 3: Chia thành L0 và R0

Sau khi áp dụng hoán vị IP, khối 64-bit kết quả sẽ được chia thành hai nửa:

L0 = 11001100 00000000 11001100 11111111

R0 = 11110000 10101010 11110000 10101010

c. Tính kết quả mở rộng R0: E[R0], với E là hàm mở rộng

Bước 1: Sử dụng bảng mở rộng E

Hàm mở rộng E trong DES sẽ lấy 32 bit của R0 và mở rộng nó thành 48 bit bằng cách sao chép và sắp xếp lại một số bit nhất định theo bảng mở rộng E.

Bảng E:

```

32 1  2  3  4  5
 4  5  6  7  8  9
 8  9 10 11 12 13
12 13 14 15 16 17
16 17 18 19 20 21
20 21 22 23 24 25
24 25 26 27 28 29
28 29 30 31 32 1

```

Mỗi số trong bảng E là vị trí của một bit trong R0. Bảng E sắp xếp lại và lặp lại một số bit để tăng từ 32 bit lên 48 bit.

Bước 2: Áp dụng bảng E trên R0

Dựa trên bảng E, ta sẽ chọn các bit từ R0 theo thứ tự được chỉ định để tạo ra E[R0].

E[R0] = 011110 100001 010101 010101 011110 100001 010101 010101

d. Tính giá trị $A = E[R0] \oplus K1$

E[R0] = 011110 100001 010101 010101 011110 100001 010101 010101

K1 = 000110 110000 001011 101111 111111 000111 000001 110010

E[R0] = 011110 100001 010101 010101 011110 100001 010101 010101

K1 = 000110 110000 001011 101111 111111 000111 000001 110010

A = 011000 010001 011110 111010 100001 100110 010100 100111

Sau khi thực hiện phép XOR, ta được:

A = 011000 010001 011110 111010 100001 100110 010100 100111

e. Chia 48-bit kết quả ở câu d và chia thành các nhóm 6 bit, thực hiện tính toán trên từng nhóm 6 bit thông qua S-box, ghi lại kết quả.

$$A = E[R0] \oplus K1$$

$$= 011000\ 010001\ 011110\ 111010\ 100001\ 100110\ 010100\ 100111$$

Bước 1: Chia thành 8 nhóm 6-bit

Chia A thành 8 nhóm 6-bit:

B1 = 011000	B5 = 100001
B2 = 010001	B6 = 100110
B3 = 011110	B7 = 010100
B4 = 111010	B8 = 100111

Bước 2: Sử dụng các bảng S-box cố định của DES

Bảng S1

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13													
1	0	15	7													
2	4	1	14													
3	15	12	8													

Bước 3: Tính toán từng nhóm 6-bit qua S-box

Kết hợp các giá trị từ S-box, ta có chuỗi 32-bit sau khi qua S-box:

$$\text{S-box output} = 01011100100001001011110010010100$$

f. Nối các kết quả tính được ở câu e thành chuỗi kết quả 32-bit, ghi lại kết quả dưới dạng binary (B).

S-box output = 0101 1100 1000 0100 1011 1100 1001 0100

→ B = 01011100100001001011110010010100

g. Tính giá trị P(B), với P là hàm hoán vị

Bảng hoán vị P: Bảng hoán vị P xác định cách sắp xếp lại các bit từ chuỗi 32-bit B:

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

Chuỗi kết quả B 32-bit từ câu f là:

B = 01011100100001001011110010010100

Áp dụng bảng hoán vị P lên B

Sau khi sắp xếp lại các bit, ta được chuỗi 32-bit kết quả P(B):

P(B) = 01011010010100010000100111001010

h. Tính giá trị **R1 = P(B) ⊕ L0**

Giá trị P(B) từ bước g:

P(B) = 01011010010100010000100111001010

Giá trị L0 từ bước b:

L0 = 11001100000000001100110011111111

P(B) = 01011010010100010000100111001010
--

L0 = 11001100000000001100110011111111

R1 = 10010110010100011100010100110101

Sau khi thực hiện phép XOR, ta được:

R1 = 10010110010100011100010100110101

i. Ghi lại kết quả ciphertext cho vòng thứ nhất

R1 = 10010110010100011100010100110101

L0 = 11001100000000001100110011111111

Trong DES, sau mỗi vòng, ciphertext tạm thời sẽ là sự nối của R1 và L0, theo thứ tự R1 trước, rồi đến L0:

Ciphertext (vòng 1) = R1 + L0

Ciphertext (vòng 1)

10010110010100011100010100110101 11001100000000001100110011111111